reference to the logger. You do that by injecting `ILogger<T>` into the `PageModel` constructor.

### Listing 12.9 Making loggers available using dependency injection

```
public class CityModel : PageModel
{
    private readonly ICityService _cityService;
    private readonly IPropertyService _propertyService;
    private readonly ILogger<CityModel> _logger;          ❶

    public CityModel(ICityService cityService,
        IPropertyService propertyService,
        ILogger<CityModel> logger)                        ❷
    {
        _cityService = cityService;
        _propertyService = propertyService;
        _logger = logger;                                 ❸
    }
```

❶ Add a field for an ILogger<T>.

❷ Inject the ILogger<T> into the constructor.

❸ Assign the parameter to the field.

The `Logger<T>` takes a generic parameter representing the category of the logger. We will look at categories in more detail soon, but for now, pass in the name of the class that uses the logger.

The next listing shows the amended `OnGetAsync` method in the `CityModel` class. In it, you use the logger instance to log a warning if the city passed in the URL is not found and an informational message if it is.

### Listing 12.10 The amended `OnGetAsync` class incorporating logging

```
public async Task<IActionResult> OnGetAsync()
{
    City = await _cityService.GetByNameAsync(Name);
    if (City == null)
    {
        _logger.LogWarning("City \"{name}\" not found", Name);    ❶
        return NotFound();
```